Chapter 5: Representative Points

Youngseo Cho

2025-12-31

Contents

1 Introduction **1**

2 Section 5.1: Uniform Design **1**
2.1 5.1.1 Uniformity 1
2.2 Monte Carlo vs Sobol vs Uniform Design 2

3 Section 5.2: Nonuniform Distributions **3**
3.1 5.2.1 Transformation Methods 3
3.2 5.2.2 Support Points 4
3.3 Correlated Normal Distribution 5

4 Section 5.2.3: Uncertainty Propagation **6**

5 Summary **8**

1 Introduction

This chapter covers uniform designs and support points for representing probability distributions.

Citation: V. R. Joseph (2026). *Experimental Design for Data Science and Engineering*. CRC Press.

```
library(SFDesign)
library(spacefillr)
library(support)
library(mnormt)
```

2 Section 5.1: Uniform Design

2.1 5.1.1 Uniformity

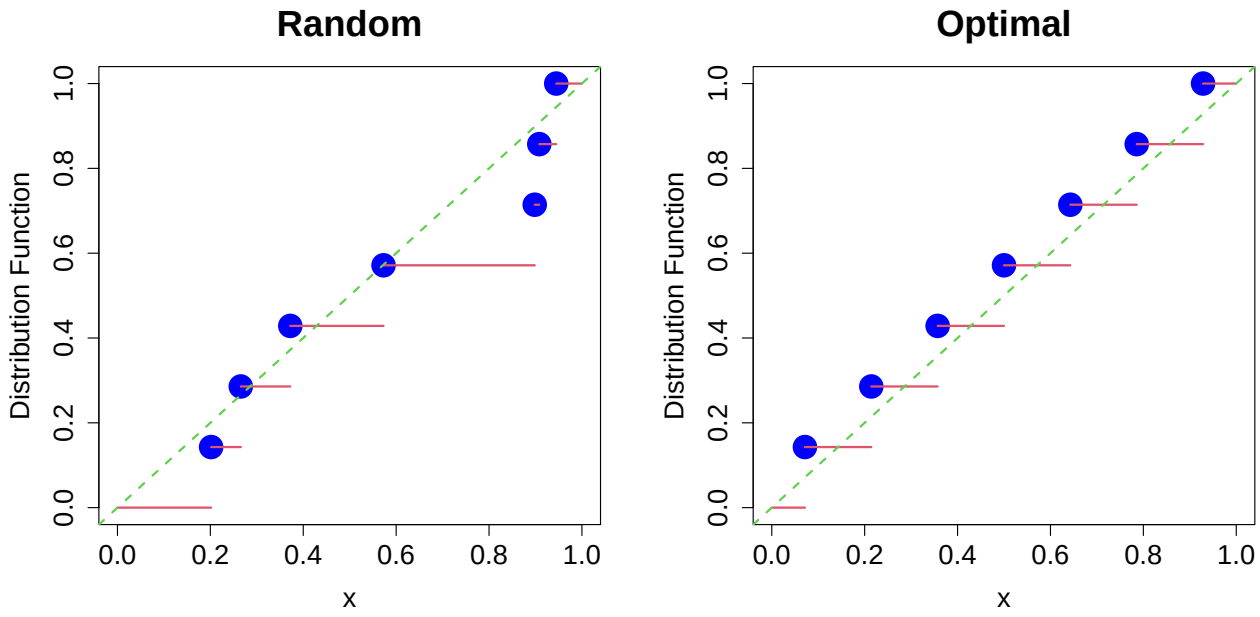
```
set.seed(1)
n <- 7
x <- sort(runif(n))

par(mar = c(5, 5, 4, 2))
par(mfrow = c(1, 2))

# Random design
d <- matrix(0, nrow = n, ncol = 2)
d[, 1] <- x
```

```r
d[, 2] <- (1:n) / n
plot(d, xlim = c(0, 1), ylim = c(0, 1), pch = 16, col = "blue", cex = 3,
     xlab = "x", ylab = "Distribution Function", main = "Random",
     cex.lab = 1.5, cex.main = 2, cex.axis = 1.5)
segments(0, 0, x[1], 0, col = 2, lwd = 2)
for (i in 1:(n-1)) segments(x[i], i/n, x[i+1], i/n, col = 2, lwd = 2)
segments(x[n], 1, 1, 1, col = 2, lwd = 2)
abline(0, 1, col = 3, lwd = 2, lty = 2)

# Optimal design
x <- ((1:n) - 0.5) / n
d <- matrix(0, nrow = n, ncol = 2)
d[, 1] <- x
d[, 2] <- (1:n) / n
plot(d, xlim = c(0, 1), ylim = c(0, 1), pch = 16, col = "blue", cex = 3,
     xlab = "x", ylab = "Distribution Function", main = "Optimal",
     cex.lab = 1.5, cex.main = 2, cex.axis = 1.5)
segments(0, 0, x[1], 0, col = 2, lwd = 2)
for (i in 1:(n-1)) segments(x[i], i/n, x[i+1], i/n, col = 2, lwd = 2)
segments(x[n], 1, 1, 1, col = 2, lwd = 2)
abline(0, 1, col = 3, lwd = 2, lty = 2)
```




```r
par(mfrow = c(1, 1))
```

2.2 Monte Carlo vs Sobol vs Uniform Design

```r
p <- 2
n <- 20
set.seed(6)

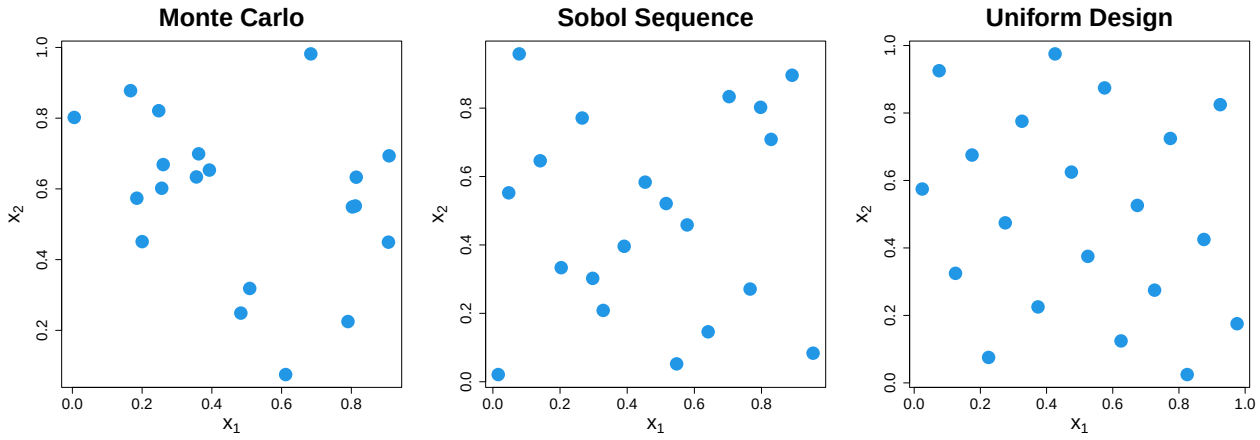
D <- uniform.optim(uniformLHD(n, p)$design)$design
R <- matrix(runif(n * 2), ncol = 2)
S <- generate_sobol_set(n, p, seed = sample(1:10000, 1))
```

```r
par(mar = c(5, 5, 4, 2))
par(mfrow = c(1, 3))

plot(R, pch = 16, col = 4, cex = 3, main = "Monte Carlo",
     cex.lab = 2, cex.main = 2.5, cex.axis = 1.5,
     xlab = expression(x[1]), ylab = expression(x[2]))

plot(S, pch = 16, col = 4, cex = 3, main = "Sobol Sequence",
     cex.lab = 2, cex.main = 2.5, cex.axis = 1.5,
     xlab = expression(x[1]), ylab = expression(x[2]))

plot(D, pch = 16, col = 4, cex = 3, main = "Uniform Design",
     cex.lab = 2, cex.main = 2.5, cex.axis = 1.5,
     xlab = expression(x[1]), ylab = expression(x[2]))
```




```r
par(mfrow = c(1, 1))
```

3 Section 5.2: Nonuniform Distributions

3.1 5.2.1 Transformation Methods

```r
p <- 2
n <- 50
set.seed(6)

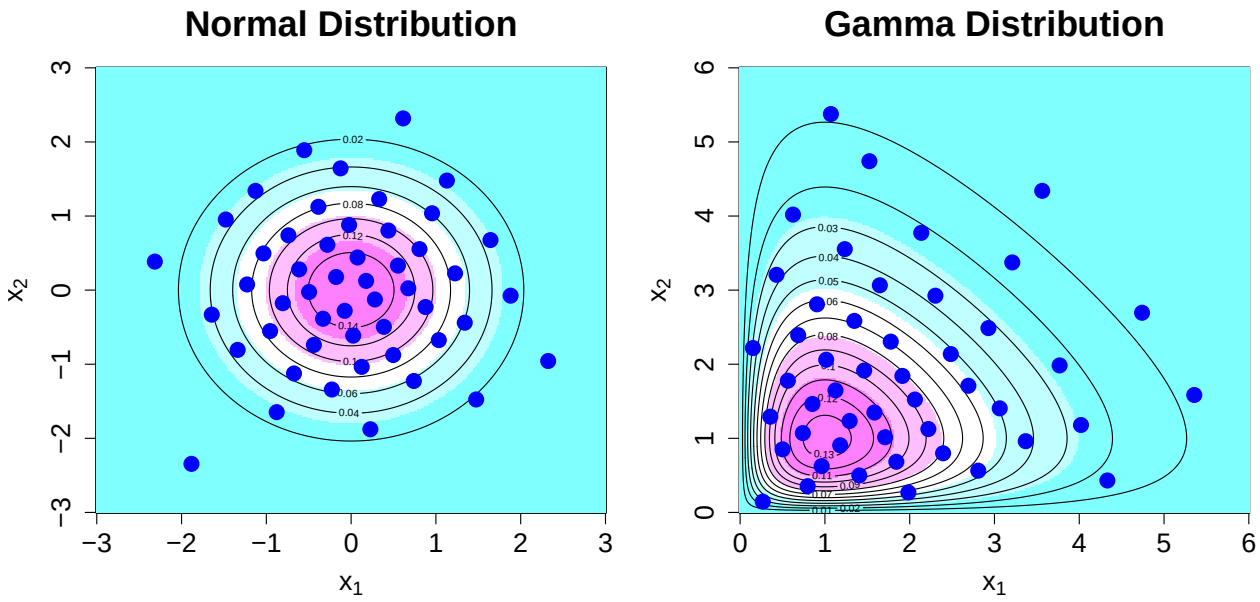
d <- uniform.optim(uniformLHD(n, p)$design)$design

par(mfrow = c(1, 2))

# Normal distribution
D <- qnorm(d)
f <- function(x) exp(-0.5 * sum(x^2)) / (2 * pi)
N.plot <- 300
p1 <- seq(-3, 3, length.out = N.plot)
p2 <- seq(-3, 3, length.out = N.plot)
fc <- matrix(apply(expand.grid(p1, p2), 1, f), N.plot, N.plot)
image(p1, p2, fc, xlab = expression(x[1]), ylab = expression(x[2]),
      col = cm.colors(5), main = "Normal Distribution",
      cex.main = 2, cex.lab = 1.5, cex.axis = 1.5)
```

```r
contour(p1, p2, fc, add = TRUE, nlevels = 10)
points(D, pch = 16, col = "blue", cex = 2)

# Gamma distribution
D <- qgamma(d, 2, 1)
f <- function(x) prod(dgamma(x, 2, 1))
p1 <- seq(0, 6, length.out = N.plot)
p2 <- seq(0, 6, length.out = N.plot)
fc <- matrix(apply(expand.grid(p1, p2), 1, f), N.plot, N.plot)
image(p1, p2, fc, xlab = expression(x[1]), ylab = expression(x[2]),
      col = cm.colors(5), main = "Gamma Distribution",
      cex.main = 2, cex.lab = 1.5, cex.axis = 1.5)
contour(p1, p2, fc, add = TRUE, nlevels = 10)
points(D, pch = 16, col = "blue", cex = 2)
```




```r
par(mfrow = c(1, 1))
```

3.2 5.2.2 Support Points

```r
set.seed(1)
n <- 50

par(mfrow = c(1, 2))

# Normal distribution
D <- sp(n, p, dist.str = rep("normal", p))$sp
```

```
## Optimizing ...
## [>                                                                ] 0 %[>
## Maximum iterations reached: done!
```

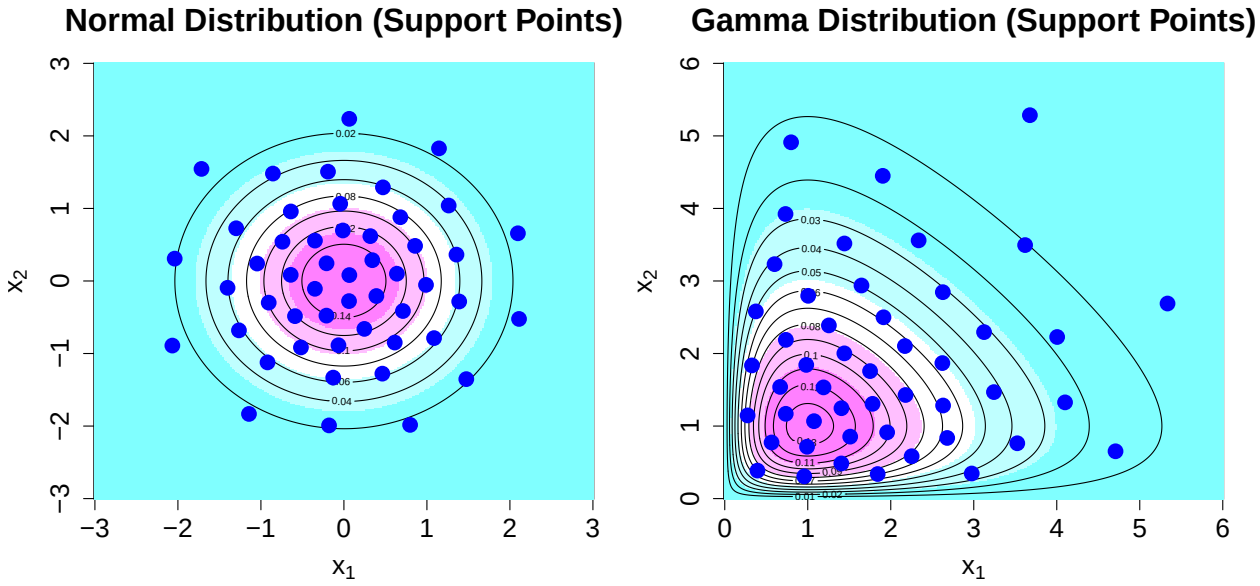
```r
f <- function(x) exp(-0.5 * sum(x^2)) / (2 * pi)
N.plot <- 300
p1 <- seq(-3, 3, length.out = N.plot)
p2 <- seq(-3, 3, length.out = N.plot)
```

```r
fc <- matrix(apply(expand.grid(p1, p2), 1, f), N.plot, N.plot)
image(p1, p2, fc, xlab = expression(x[1]), ylab = expression(x[2]),
      col = cm.colors(5), main = "Normal Distribution (Support Points)",
      cex.main = 1.8, cex.lab = 1.5, cex.axis = 1.5)
contour(p1, p2, fc, add = TRUE, nlevels = 10)
points(D, pch = 16, col = "blue", cex = 2)

# Gamma distribution
dist.param <- vector("list", p)
for (l in 1:p) dist.param[[l]] <- c(2, 1)
D <- sp(n, p, dist.str = rep("gamma", p), dist.param = dist.param)$sp
```

```
## Optimizing ...
## [>                                                                  ] 0 %[>
## Maximum iterations reached: done!
```

```r
f <- function(x) prod(dgamma(x, 2, 1))
p1 <- seq(0, 6, length.out = N.plot)
p2 <- seq(0, 6, length.out = N.plot)
fc <- matrix(apply(expand.grid(p1, p2), 1, f), N.plot, N.plot)
image(p1, p2, fc, xlab = expression(x[1]), ylab = expression(x[2]),
      col = cm.colors(5), main = "Gamma Distribution (Support Points)",
      cex.main = 1.8, cex.lab = 1.5, cex.axis = 1.5)
contour(p1, p2, fc, add = TRUE, nlevels = 10)
points(D, pch = 16, col = "blue", cex = 2)
```



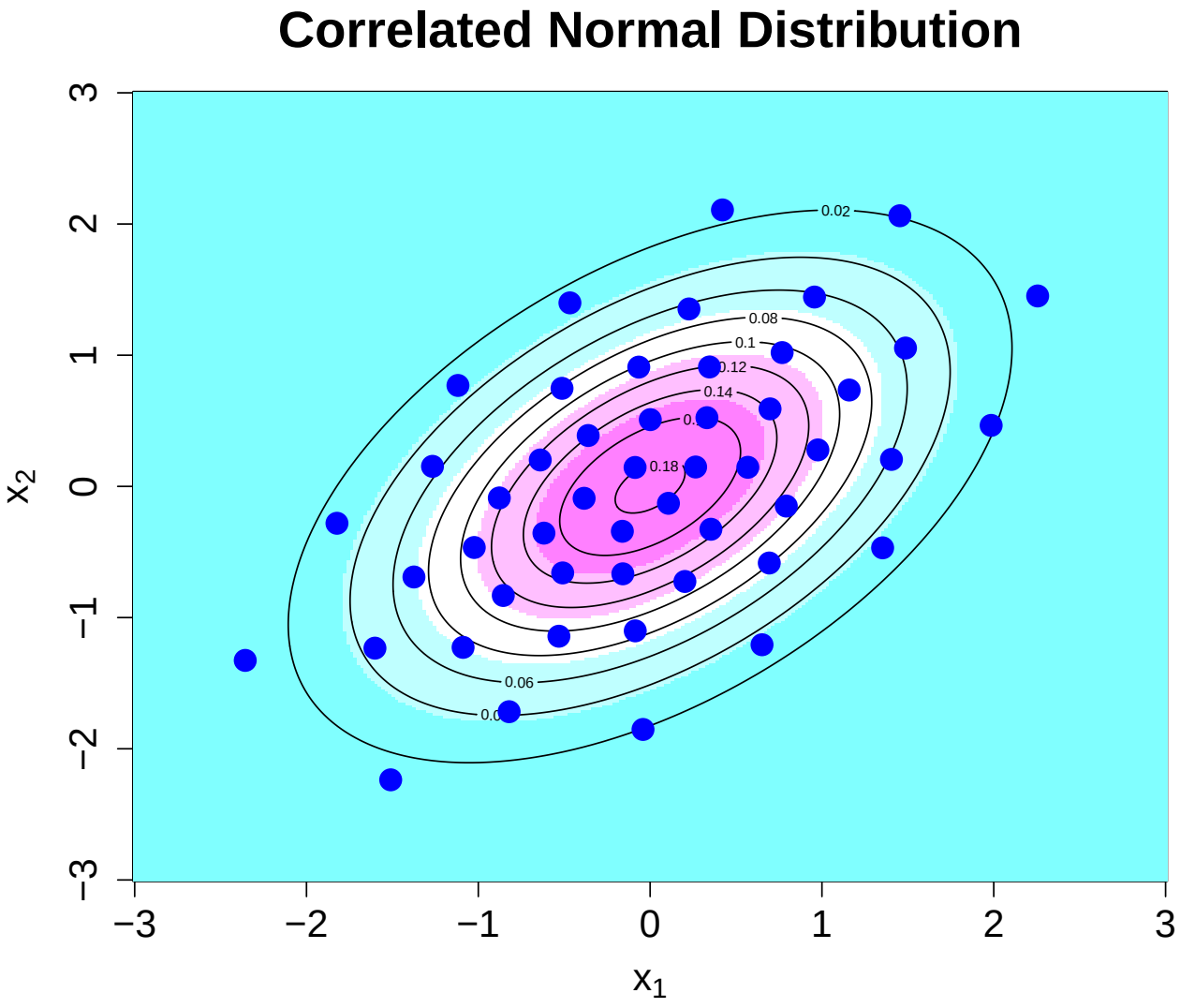

```r
par(mfrow = c(1, 1))
```

3.3 Correlated Normal Distribution

```r
set.seed(1)
n <- 50
N <- 10000
X <- rmnorm(N, mean = c(0, 0), varcov = matrix(c(1, 0.5, 0.5, 1), 2, 2))
D <- sp(n, p, dist.samp = X)$sp
```

```
## Optimizing ...
## [>                                                                  ] 0 %[>
## Maximum iterations reached: done!
```

```r
f <- function(x) dmnorm(x, mean = c(0, 0), varcov = matrix(c(1, 0.5, 0.5, 1), 2, 2))
N.plot <- 300
p1 <- seq(-3, 3, length.out = N.plot)
p2 <- seq(-3, 3, length.out = N.plot)
fc <- matrix(apply(expand.grid(p1, p2), 1, f), N.plot, N.plot)
image(p1, p2, fc, xlab = expression(x[1]), ylab = expression(x[2]),
      col = cm.colors(5), main = "Correlated Normal Distribution",
      cex.main = 2, cex.lab = 1.5, cex.axis = 1.5)
contour(p1, p2, fc, add = TRUE, nlevels = 10)
points(D, pch = 16, col = "blue", cex = 2)
```



4 Section 5.2.3: Uncertainty Propagation

```r
# Borehole function
f <- function(x) {
```

```
  val <- 2 * pi * x[3] * (x[4] - x[6]) /
    (log(x[2] / x[1]) * (1 + 2 * x[7] * x[3] / (log(x[2] / x[1]) * x[1]^2 * x[8]) + x[3] / x[5]))
  return(val)
}

p <- 8
lower <- c(0.05, 100, 63070, 990, 63.1, 700, 1120, 9855)
upper <- c(0.15, 50000, 115600, 1110, 116, 820, 1680, 12045)

set.seed(1)
N <- 100000

# Generate input samples
X1 <- 0.1 + 0.01618 * rnorm(N)
X2 <- exp(7.71 + 1.0056 * rnorm(N))
X <- matrix(0, nrow = N, ncol = p)
X[, 1:2] <- cbind(X1, X2)
X[, 3:p] <- matrix(runif(N * 6), nrow = N)
X[, 3:p] <- sweep(X[, 3:p], 2, upper[3:p] - lower[3:p], "*")
X[, 3:p] <- sweep(X[, 3:p], 2, lower[3:p], "+")
true <- apply(X, 1, f)

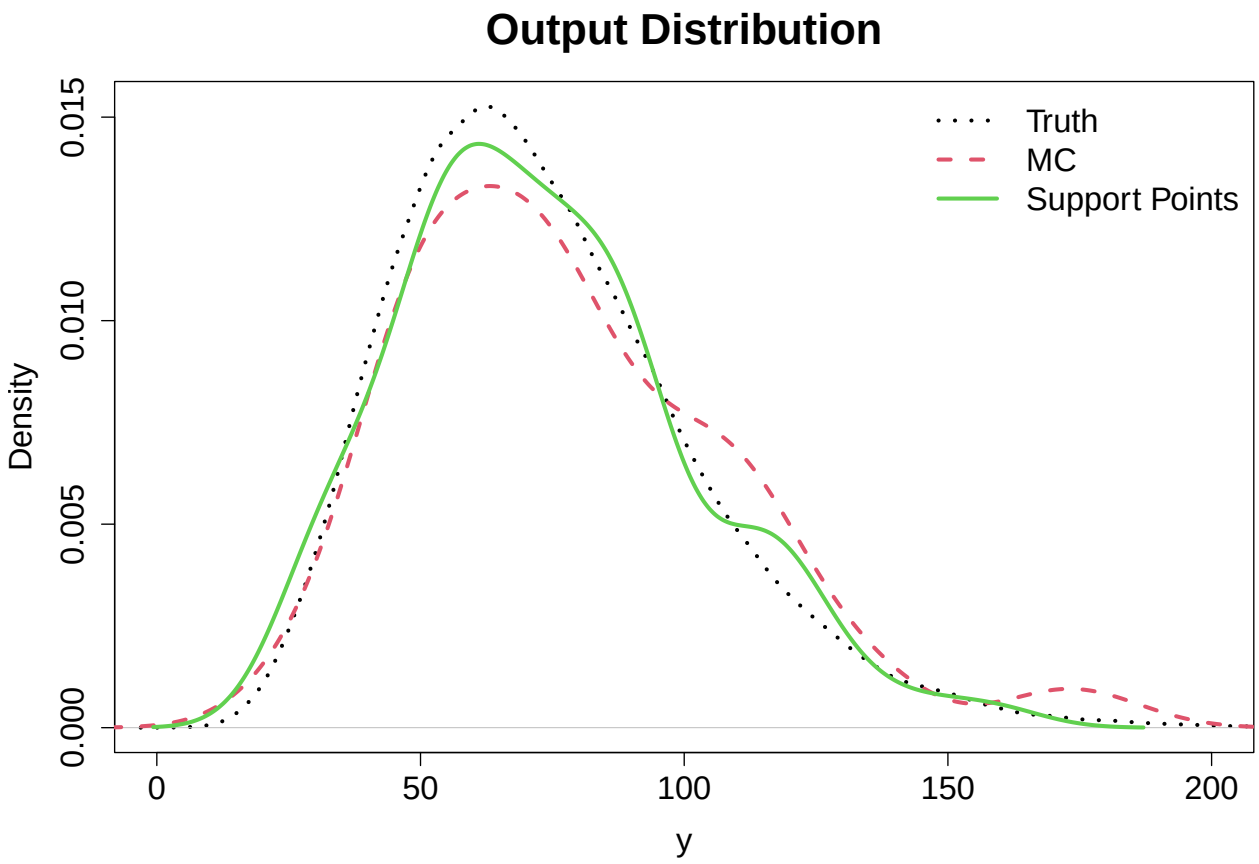
# Monte Carlo
n <- 100
x <- X[sample(1:N, n), ]
mc <- apply(x, 1, f)

# Support points
mu <- apply(X, 2, mean)
sigma <- apply(X, 2, sd)
Z <- sweep(X, 2, mu, "-")
Z <- sweep(Z, 2, sigma, "/")
D <- sp(n, p, dist.samp = Z)$sp
```

```
## Optimizing ...
## [>                                                                  ] 0 %[>
## Maximum iterations reached: done!
```

```
D <- sweep(D, 2, sigma, "*")
D <- sweep(D, 2, mu, "+")
y <- apply(D, 1, f)

plot(density(true), lty = 3, lwd = 3, xlab = "y", main = "Output Distribution",
     cex.main = 2, cex.lab = 1.5, cex.axis = 1.5, xlim = c(0, 200))
lines(density(mc), col = 2, lty = 2, lwd = 3)
lines(density(y), col = 3, lwd = 3)
legend("topright", legend = c("Truth", "MC", "Support Points"),
       col = c(1, 2, 3), lwd = 3, lty = c(3, 2, 1), bty = "n", cex = 1.5)
```



5 Summary

This chapter covered:

1. **Uniform designs** - Minimizing discrepancy for uniform distributions
2. **Quasi-Monte Carlo** - Sobol sequences for low-discrepancy sampling
3. **Transformation methods** - Converting uniform designs to other distributions
4. **Support points** - Energy-distance based representative points for arbitrary distributions
5. **Uncertainty propagation** - Using support points for efficient output distribution estimation